

ASCL HANDS-ON TRAINING · DAY 2 OF 4

ROS2

ROS 2 & LIMO Cobot Training

From Robot Concepts to Real-World Action

LAB	Automated Systems & Computing Lab (ASCL)
LAB LEADER	Prof. Ahmad Taher Azar

Dr Saim Ahmed
Trainer 1 · Day 1 & Day 3

Eng. Zeeshan Haider
Trainer 2 · Day 2 & Day 4



ASCL
Automated Systems & Computing Lab

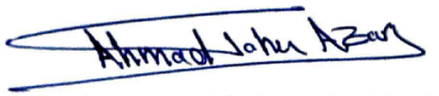
جامعة الامير سلطان
PRINCE SULTAN
UNIVERSITY



ACKNOWLEDGMENTS

This Training is supported by a research grant funded by the Research, Development, and Innovation Authority (RDIA), Kingdom of Saudi Arabia, with grant number 13382-psu-2023- PSNU-R-3-1-EI.

Prof. Ahmad Taher Azar



Full Professor, Leader of Automated Systems & Computing Lab (ASCL)

TRAINING AGENDA · 4 DAYS

Table of Contents

TRAINING 1 · DAYS 1–2 · FUNDAMENTALS

01 **Day 1 · What is ROS & ROS2?**

- Sense → Think → Act & the group-chat model
- Core concepts: nodes, topics, messages, services, actions
- turtlesim hands-on lab

02 **Day 2 · ROS2 Hands-On: CLI & First Nodes**

- The `ros2` CLI: node · topic · service · param
- Workspaces & packages
- First publisher & subscriber in Python

TRAINING 2 · DAYS 3–4 · ON THE ROBOT

03 **Day 3 · Meet the LIMO Cobot: Hardware & Bring-Up**

- Hardware, motion modes & sensors
- Connect & bring-up: NoMachine + chassis driver
- Drive & sense

04 **Day 4 · Applications: SLAM, Navigation & the Arm**

- SLAM mapping with Cartographer
- Navigation with Nav2
- myCobot 280 arm control

REFRESHER · DAY 1 IN 60 SECONDS

The building blocks — what, when, why

🧩 Node

One program, one job

WHEN you need a capability

WHY isolated — one crash doesn't stop the rest

📡 Topic msg

A stream, nobody waits

WHEN live data keeps flowing

WHY decoupled — any number of readers

☎️ Service srv

Ask once → one reply

WHEN a quick, exact task

WHY one immediate answer

🎯 Action action

Long job, watch & cancel

WHEN work that takes time

WHY feedback while it runs

⚙️ Parameter

A node's own setting

WHEN tune it live

WHY no rebuild, no restart

📄 Topics & services carry a **typed msg** — the contract is always **name + type**.

Pick in 3 seconds: nothing comes back? → topic. One quick answer? → service. Takes a while? → action. Just a setting? → parameter.

REFRESHER · HOW A NODE CONNECTS — THE FOUR CHANNELS

How a node connects — topics, services, actions & parameters

🧩 **NODE** `/demo_node`

one program, one job — every connection below hangs off it

📡 **Topic** `/chatter`

pub/sub — a steady stream, nobody waits

☎️ **Service** `/add_two_ints`

request → one reply, you wait for it

🎯 **Action** `/navigate_to_pose`

long job — progress fed back, cancellable

⚙️ **Parameter** `speed = 0.5`

the node's **own setting** — not a conversation

demo_node.py — one node, all four connections

```python
class DemoNode(Node):                    # 🧩 one program, one job
    def __init__(self):
        super().__init__('demo_node')

        # ⚙️ PARAMETER — the node's own setting
        self.declare_parameter('speed', 0.5)

        # 📡 TOPIC — publish data out
        self.pub = self.create_publisher(
            String, '/chatter', 10)
        # 📡 TOPIC — subscribe to data in
        self.sub = self.create_subscription(
            String, '/status', self.on_status, 10)

        # ☎️ SERVICE — answer one request
        self.srv = self.create_service(
            AddTwoInts, '/add_two_ints', self.add)

        # 🎯 ACTION — start a long job
        self.nav = ActionClient(
            self, NavigateToPose, '/navigate_to_pose')
```

Connecting = calling the create_* function once. One node owns all four at the same time — a **parameter** is its own setting, the other three are conversations with other nodes. `ros2 node info /demo_node` lists them all.

REFRESHER · DAY 1 TURTLESIM LAB (1 OF 2) — RUN IT & DRIVE IT

Get the turtle running — 60 seconds

Every terminal starts with `source /opt/ros/jazzy/setup.bash` — **1** the turtle · **2** the driver · **3** the detective.

1 · Start the "robot" Terminal 1

run first — leave it running

```
ros2 run turtlesim turtlesim_node
```

A turtle appears in a window. It publishes `/turtle1/pose` and listens for speed commands on `/turtle1/cmd_vel`.

2 · Drive it Terminal 2

run second

```
ros2 run turtlesim turtle_teleop_key
```

Click this terminal, then press ↑ ↓ ← →. Teleop publishes a `Twist` on `/turtle1/cmd_vel`; turtlesim subscribes — two nodes, one topic.

3 · Who's running? 🔍 Terminal 3 — the detective

list what is alive, then what travels on it

```
ros2 node list                    # /turtlesim · teleop_turtle
ros2 topic list                   # /turtle1/cmd_vel · /turtle1/pose
ros2 topic info /turtle1/cmd_vel  # type Twist + pub/sub counts
```

Look inside — then take control

4 · Watch the data echo

Terminal 3 — subscribe & print

```
ros2 topic echo /turtle1/cmd_vel
```

Prints every message live — press the arrows and watch `linear.x` / `angular.z` change. `Ctrl+C` to stop.

5 · Move it without teleop topic pub

Terminal 3 — publish a Twist directly

```
ros2 topic pub -r 10 /turtle1/cmd_vel \
  geometry_msgs/msg/Twist \
  "{linear: {x: 0.5}, angular: {z: 0.1}}"
```

The turtle circles by itself — ten `Twist` messages per second, no keyboard. `Ctrl+C` to stop.

6 · Add a second turtle service

Terminal 3 — one request, one response

```
ros2 service call /spawn \
  turtlesim/srv/Spawn \
  "{x: 2, y: 2, theta: 0.2, name: 'turtle2'}"
```

One request → one response: a **service**, answered once — not a stream.

7 · Drive turtle2 remap

Terminal 4 — same node, new target

```
ros2 run turtlesim turtle_teleop_key \
  --ros-args --remap turtle1/cmd_vel:=turtle2/cmd_vel
```

The **same** teleop node, pointed at the other turtle by name — no code change, just a remap.

See it all at once: `rqt_graph` — nodes and topic in one picture, exactly the Day-1 diagram.

ROADMAP

Today's plan

01
ROS2 CLI Tools
List, echo, measure, call, inspect messages — the five superpowers. 10 min

02
Workspace & Packages
Where your ROS2 code lives. 10 min

03
Publisher & Subscriber
Write a `talker` and `listener` in Python.
30 min

No Python experience needed — every line is explained; code is copy-paste ready.

BY THE END OF TODAY

You will be able to…

- Use `ros2 node` / `topic` / `interface` / `service` / `param` to inspect a live system
- **Echo** a topic and measure its frequency with `ros2 topic hz`
- Create a **workspace** and a **package**, build with `colcon build`
- Write and run a **publisher** and a **subscriber** in Python
- Explain **why** two nodes find each other with zero configuration

01

ROS2 CLI Tools

Be the detective — interrogate a running system.

IF YOU'RE NEW TO THE TERMINAL

5-minute survival kit

Action	Command	Meaning
Run a command	`ros2 node list`	Type it, press **Enter**
Stop a running program	`Ctrl+C`	Stop whatever runs in this terminal
New terminal	`Ctrl+Alt+T`	Open another (you'll need several)

That's all you need today. Copy-paste everything else.

ONE TOOL, FIVE SUPERPOWERS

The `ros2` CLI

The pattern is always the same:

```pattern
ros2 <thing> <action>      # e.g.  ros2 topic list
```

Superpower	Command family	Asks...
👥 Nodes	`ros2 node`	"What programs are running?"
📡 Topics	`ros2 topic`	"What data is flowing?"
☎️ Services	`ros2 service`	"What can I ask the system?"
⚙️ Parameters	`ros2 param`	"What settings are stored?"
📄 Messages	`ros2 interface`	"What's inside the data?"

> Stuck? `ros2 <thing> --help` always lists what a command family can do.

STEP 0

Start the system to interrogate

Terminal 1 — leave it running

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtlesim_node
```

Keep this terminal alive — it's the turtle we'll investigate.

STEP 1 · WHO'S RUNNING?

`ros2 node`

List nodes

Terminal 2

```
ros2 node list
```

→ `/turtlesim`

Node résumé

Terminal 2

```
ros2 node info /turtlesim
```

→ its publishers, subscribers, services, params

Notice `/turtle1/pose` under **Publishers** and `/turtle1/cmd_vel` under **Subscribers** — exactly Day 1.

STEP 2 · WHAT'S FLOWING?

`ros2 topic` & `ros2 interface`

1 · List topics

Terminal 2

```
ros2 topic list
ros2 topic info /turtle1/pose
```

→ message type + publisher/subscriber counts

2 · Topic → msg type

Terminal 2

```
ros2 topic type /turtle1/pose
```

→ `turtlesim/msg/Pose` — the topic's message type

3 · Peek inside the msg

Terminal 2

```
ros2 interface show \
  turtlesim/msg/Pose
```

→ fields: `x`, `y`, `theta`, velocities

Every topic carries **exactly one message type** — the **msg**. Publisher & subscriber must agree on the topic name **and** the msg type. That's the contract.

STEP 3 · LIVE DATA + FREQUENCY

Watch it & measure it

Watch live

echo

```
ros2 topic echo /turtle1/pose
```

Streams position — `Ctrl+C` to stop.

Measure rate

hz

```
ros2 topic hz /turtle1/pose
```

→ `average rate: 62.5` (~60 Hz)

Why it matters: the LIMO's LiDAR publishes `/scan` at ~6 Hz. `ros2 topic hz` is your first tool when "the robot feels sluggish."

STEP 4 · ASK & CONFIGURE

`ros2 service` & `ros2 param`

Call a service

Terminal 2

```
ros2 service list
ros2 service call /clear \
  std_srvs/srv/Empty
```

→ the trail disappears — request answered by response.

Change a setting

Terminal 2

```
ros2 param list
ros2 param get /turtlesim \
  background_b
ros2 param set /turtlesim \
  background_b 0
```

→ window changes color instantly — no restart.

RECAP

Your daily detective kit

Command you ran	Plain meaning
`ros2 node list` / `node info`	Who's running, and what can they do?
`ros2 topic list` / `info` / `type` / `echo` / `hz`	What flows, how fast, what msg type, what's inside?
`ros2 interface show`	What fields does a message (msg) carry?
`ros2 service list` / `call`	What can I ask, and what happens?
`ros2 param list` / `get` / `set`	What settings exist, can I change them live?

Troubleshooting always starts here: is the node alive? → is data flowing? → is the data sensible?

02

Workspace & Packages

Where your ROS2 code lives — the folder anatomy you'll use every day.

THE BIG PICTURE

Workspace → Package → Code

what you'll have after the lab — ~/ros2_ws

```
ros2_ws/                  📦 the workspace — one folder for everything
├── src/                  ✍️ you write code here
│   └── my_first_pkg/     a package = one project
│       ├── package.xml       its ID card
│       ├── CMakeLists.txt    build rules
│       ├── setup.py          how to install your Python
│       └── my_first_pkg/     your node files: talker.py, listener.py
├── build/                auto — build leftovers
├── install/              auto — the runnable copy
└── log/                  auto — build diary
```

Workspace
one folder holding all your projects

Package
one project inside it — one book on the shelf

Code
the node files inside the package — your program

`src/` → `colcon build` → `install/` → `ros2 run`

Write in `src/`, run from `install/`. Everything else is generated — never edit it by hand.

THE FOLDER ANATOMY

What each folder is for

Folder	Analogy	Contents
`ros2_ws/`	Your project folder	Everything
`src/`	Your drafts	Source code you write
`build/`	The workshop floor	Compiled intermediates (auto)
`install/`	The finished shelf	Ready-to-run programs (auto)
`log/`	The diary	Build logs (auto)
a **package** in `src/`	One book project	One or more nodes + metadata

> Rule of thumb: you write in `src/`, you run from `install/` — never edit `build/`, `install/`, or `log/`.

STEP 1

Create the workspace

Terminal

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

- `mkdir -p` = make directory, including parents (creates both `ros2_ws` and `src`)
- `cd` = change directory — step inside

STEP 2

Create a package

Terminal

```
ros2 pkg create my_first_pkg --build-type ament_python \
  --dependencies rclpy
```

`--dependencies rclpy` says "this package uses the Python client library" — `rclpy` is what every Python node imports to talk to ROS2.

Created file	Purpose
`package.xml`	The package's ID card (name, version, dependencies)
`CMakeLists.txt`	Build instructions (present even for Python)
`my_first_pkg/`	Where your Python code goes
`setup.py`	Tells ROS2 how to install your Python code

STEP 3

Build & source

Terminal

```
cd ~/ros2_ws
colcon build --packages-select my_first_pkg
ls ~/ros2_ws          # now: build/ install/ log/ src/
source install/setup.bash
```

Why the source line matters: it tells ROS2 "my new package exists." "package not found" almost always means you forgot to `source install/setup.bash`.

Pro tip: `echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc` auto-sources every new terminal.

STEP 4

Is my package visible?

Terminal

```
ros2 pkg list | grep my_first_pkg
```

If you see `my_first_pkg` → built, installed, and found. 🎉

RECAP

Three commands + one source line

The pattern

- `mkdir -p ~/ros2_ws/src`
- `ros2 pkg create …`
- `colcon build`
- `source install/setup.bash`

One-liners

- **Workspace** = one big folder for all projects
- **Package** = one project (a book on the shelf)
- `colcon build` = compile `src/` → `install/`
- `source …` = "ROS2, notice my package"

The LIMO's own packages (`limo_base`, `limo_bringup`) follow this exact pattern.

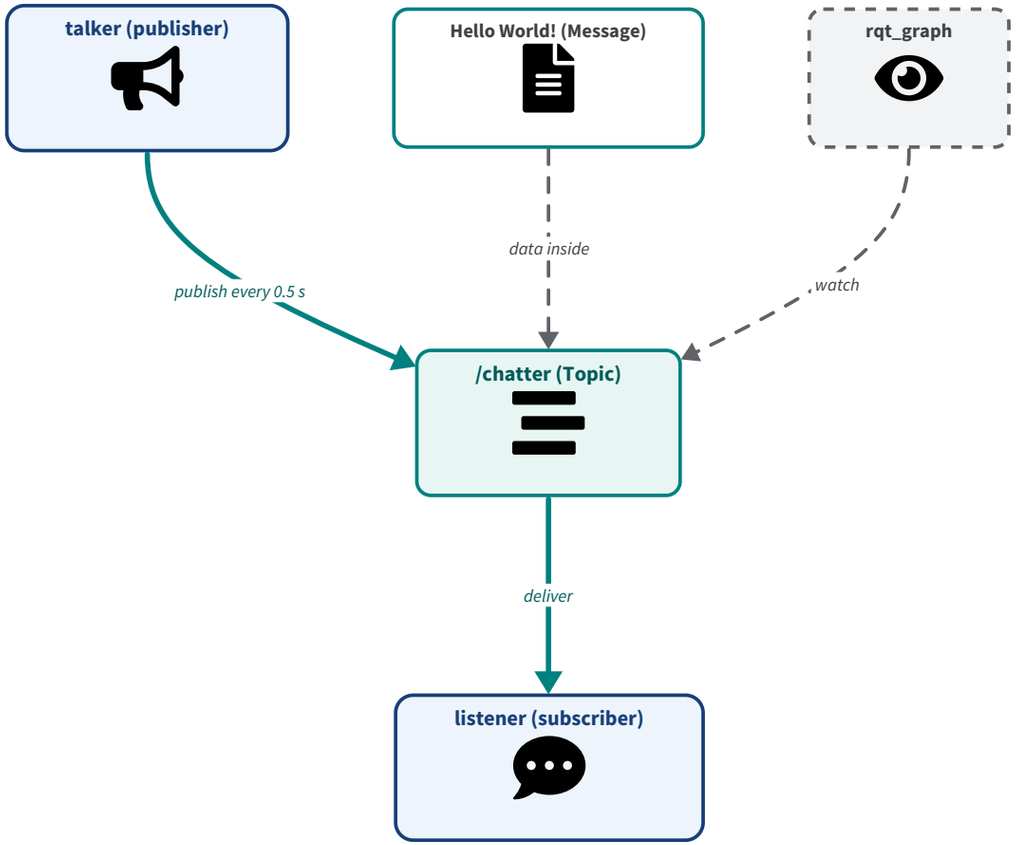
03

First Publisher & Subscriber

Your code talks — the "Hello World" of ROS2.

THE GOAL IN ONE PICTURE

talker → /chatter → listener



`talker` publishes "Hello World" to `/chatter` every 0.5 s · `listener` subscribes · zero configuration

STEP 1

Create the node files

Terminal — go to the right place

```
cd ~/ros2_ws/src/my_first_pkg/my_first_pkg
```

Two files to create: `talker.py` (publisher) and `listener.py` (subscriber).

FILE 1 — TALKER.PY

The publisher

talker.py — save & copy-paste

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String


class Talker(Node):
    def __init__(self):
        super().__init__('talker')                # node name → rqt_graph
        self.pub = self.create_publisher(String, '/chatter', 10)
        self.timer = self.create_timer(0.5, self.tick)   # every 0.5 s


    def tick(self):
        msg = String()
        msg.data = 'Hello World!'
        self.pub.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
```

READ IT LINE BY LINE

What the talker does

1. `create_publisher(String, '/chatter', 10)` → "I'll send `String` messages on `/chatter`, queue 10"
2. `create_timer(0.5, self.tick)` → "run `tick()` twice per second"
3. `tick()` → build a message, publish it, log it
4. `rclpy.spin(Talker())` → "keep this node alive, processing messages"

FILE 2 — LISTENER.PY

The subscriber

listener.py — save & copy-paste

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String


class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        # subscribe to the same topic, same type
        self.sub = self.create_subscription(
            String, '/chatter', self.on_msg, 10)


    def on_msg(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)


def main(args=None):
    rclpy.init(args=args)
```

THE MAGIC

Discovery is automatic

The listener never mentions the talker. It only says **"I listen on `/chatter`."**

- Talker starts first → listener finds it.
- Listener starts first → talker finds it.

The entire contract is the topic name. That's ROS.

STEP 2

Tell the package these scripts exist

edit setup.py — entry_points

```
entry_points={
    'console_scripts': [
        'talker = my_first_pkg.talker:main',
        'listener = my_first_pkg.listener:main',
    ],
},
```

Pattern: `'command_name = package.file:function'`. Save (`Ctrl+O`, Enter) and exit (`Ctrl+X`).

STEP 3

Rebuild & source

Terminal

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

Build reads `src/`, writes `install/`; the source line makes the new commands visible.

STEP 4

Run the show — 2 terminals

Terminal 1 — listener

runs first

```
source ~/ros2_ws/install/setup.bash
ros2 run my_first_pkg listener
```

It waits silently... listening.

Terminal 2 — talker

runs second

```
source ~/ros2_ws/install/setup.bash
ros2 run my_first_pkg talker
```

Watch Terminal 1: `I heard: "Hello World!"` ×2/sec 🎉

STEP 5

See it as a picture

Terminal 3

```
rqt_graph
```

`talker` and `listener` (ellipses) connected through `/chatter` (rectangle) — the pubsub diagram, but real and built by you.

STEP 6 · BONUS

Interrogate your own creation

Terminal 3 — Day 2's detective tools

```
ros2 node list           # now shows talker and listener!
ros2 topic hz /chatter   # ~2 Hz — matching the 0.5 s timer
ros2 topic type /chatter # std_msgs/msg/String — your msg
```

RECAP

What each piece really did

Piece of code	What it really did
`create_publisher(String, '/chatter', 10)`	"I broadcast String messages on channel /chatter"
`create_timer(0.5, …)`	"Do the next thing twice per second"
`create_subscription(String, '/chatter', …)`	"I tune in to channel /chatter"
The topic name matching	The ONLY thing that connects them

THE BIG TAKEAWAY

Same pattern, real robot

The talker and listener never knew about each other — they agreed on a **topic name**. That's the entire contract.

On the LIMO: `teleop_twist_keyboard` publishes `/cmd_vel` → `limo_base` subscribes. **Same pattern, real robot.**



Quick Check

Five minutes. Pass ≥ 70% — answers revealed on each slide.

Q1 · Which command shows ALL running nodes?

A) `ros2 topic list`

B) `ros2 node info /turtlesim`

C) `ros2 node list`

D) `rqt_graph`

▸ Show answer

Q2 · LiDAR data seems slow. Which command measures the rate?

A) `ros2 topic echo /scan`

B) `ros2 topic info /scan`

C) `ros2 topic hz /scan`

D) `ros2 node list`

▸ Show answer

Q3 · How do you see which message type (msg) a topic carries?

A) `ros2 node info /turtlesim`

B) `ros2 topic type /turtle1/pose`

C) `ros2 topic hz /turtle1/pose`

D) `rqt_graph`

▸ **Show answer**

Q4 · After creating a package, what makes it visible to `ros2 run`?

A) `ros2 pkg list`

B) `colcon build` only

C) `colcon build` then `source install/setup.bash`

D) Restarting the computer

▸ **Show answer**

Q5 · In the talker, what does this line do?

talker.py

```
self.pub = self.create_publisher(String, '/chatter', 10)
```

A) Subscribes to /chatter and prints messages

B) Creates a publisher sending String messages on /chatter

C) Creates a new node named /chatter

D) Publishes one message immediately

▸ Show answer

Q6 · The listener never mentions the talker. How do they connect?

A) They share the same IP address

B) The talker tells the listener its address

C) They both use the same topic name — discovery is automatic

D) You must configure connections by hand

▸ **Show answer**

Q7 · `ros2 run my_first_pkg talker` → "package not found". Fix?

A) Delete the package and start over

B) `cd ~/ros2_ws` only

C) Re-run `source install/setup.bash` (rebuild first)

D) Use `ros2 run talker`

▸ **Show answer**

Q8 · What does this `setup.py` entry do?

```setup.py
'talker = my_first_pkg.talker:main',
```

A) Renames the file talker.py

B) Creates a runnable command `talker` → `main()` in talker.py

C) Publishes to a topic named talker

D) Sets the node's name to my_first_pkg

▸ **Show answer**

MINI-DESIGN CHALLENGE

Publish a `Twist` on `/cmd_vel`

Write a node that publishes the robot's speed (`geometry_msgs/msg/Twist`) on `/cmd_vel` once per second — using the talker pattern.

The two changes

diff vs talker.py

```
from geometry_msgs.msg import Twist
...
self.pub = self.create_publisher(
    Twist, '/cmd_vel', 10)
self.timer = self.create_timer(
    1.0, self.tick)
...
msg = Twist()
msg.linear.x = 0.1    # forward 0.1 m/s
```

Twice per second?

Change `create_timer(1.0, …)` → `create_timer(0.5, …)`.

> **This is literally how the LIMO's teleop works** — publish a `Twist` on `/cmd_vel`, and `limo_base` moves the robot.

DAY 2 COMPLETE ✅

Tomorrow: the real robot.

Day 3 introduces the **LIMO Cobot** — connect over NoMachine, start its brain, and **drive it yourself** with the exact `/cmd_vel` pattern you wrote today.